

机器人综合实验报告

课程名称：机器人综合实验

姓名：_____

学号：_____

班级：_____

指导教师：_____

2026 年 6 月

目录

摘要	1
1 实验一：开发环境配置	2
1.1 实验目的	2
1.2 软件环境	2
1.3 硬件连接与安全约束	2
1.4 仓库结构与实验代码组织	3
1.5 实验结果与小结	3
2 实验二：机械臂末端直线运动控制	4
2.1 实验目的	4
2.2 机械臂运动学模型	4
2.3 直线轨迹规划方法	4
2.4 程序实现	5
2.5 仿真与实验结果	6
2.6 结果分析与小结	7
3 实验三：机械臂抓取与搬运	8
3.1 实验目的	8
3.2 任务流程设计	8
3.3 关键位姿与夹爪控制	8
3.4 程序实现	9
3.5 实验结果与分析	10
3.6 小结	11
4 实验四：相机标定	11
4.1 实验目的	11
4.2 相机成像模型	11
4.3 内参与畸变标定	11
4.4 工作空间坐标转换	13
4.5 标定结果与分析	14

目录	II
4.6 小结	15
5 实验五：基于颜色识别的视觉抓取	15
5.1 实验目的	15
5.2 颜色分割与目标定位	15
5.3 像素坐标到机械臂坐标转换	17
5.4 抓取流程设计	18
5.5 实验结果与分析	18
5.6 小结	19
6 实验六：基于 YOLO 的视觉抓取	20
6.1 实验目的	20
6.2 数据集与类别设计	20
6.3 YOLO 模型训练与推理	20
6.4 与颜色识别方法的接口复用	22
6.5 实验结果与对比分析	24
6.6 小结	24
7 总结与展望	25
7.1 实验总结	25
7.2 误差来源分析	25
7.3 改进方向	25

摘要

本文围绕本学期机器人综合实验展开，按照“环境配置、运动学建模、轨迹控制、抓取搬运、相机标定、视觉定位抓取、YOLO 目标识别”的技术路线，整理了六组实验的实验目的、理论基础、程序实现、实验过程与结果分析。实验对象为六自由度桌面机械臂，软件环境基于 Python、Robotics Toolbox、OpenCV 与 YOLO 相关工具链，硬件控制通过串口转 CAN 总线完成。

通过前半部分实验，完成了机械臂开发环境配置、DH 参数建模、正逆运动学验证以及末端直线轨迹规划；通过中间实验，实现了基于关键位姿序列的物体抓取与搬运；通过后半部分实验，完成相机内参与工作空间坐标转换标定，并分别采用颜色分割与 YOLO 检测方法实现视觉引导抓取。最终实验形成了从机械臂基础运动控制到视觉感知闭环抓取的完整流程。

关键词：六自由度机械臂；DH 参数；轨迹规划；相机标定；颜色识别；YOLO；视觉抓取

1 实验一：开发环境配置

1.1 实验目的

本实验的目标是完成机械臂综合实验的软件与硬件基础环境配置，为后续运动控制、视觉感知和抓取实验提供统一的运行平台。环境配置不仅包括 Python 依赖安装，还包括机械臂串口通信方式、仿真模型文件、实验代码目录和安全运行约束的确认。

1.2 软件环境

本实验仓库根目录提供了 `environment.yml`，用于创建名为 `robot` 的 conda 环境。环境中 Python 版本为 3.10.20，主要依赖包括：

表 1: 实验环境主要软件依赖

类别	软件包	用途
机器人建模	<code>roboticstoolbox-python</code> , <code>spatialmath-python</code>	DH 建模、正逆运动学、轨迹规划
数值计算	<code>numpy</code> , <code>scipy</code> , <code>modern-robotics</code>	矩阵计算、优化求解
视觉处理	<code>opencv-python</code> , <code>opencv-contrib-python</code>	相机标定、颜色分割、图像显示
仿真与显示	<code>matplotlib</code> , <code>pybullet</code> , <code>trimesh</code>	轨迹可视化、模型检查
深度学习	<code>torch</code> , <code>torchvision</code>	YOLO 训练和推理基础
通信控制	<code>pyserial</code>	串口转 CAN 通信

首次配置时可在仓库根目录执行：

Listing 1: conda 环境创建命令

```
1 conda env create -f environment.yml
2 conda activate robot
```

后续运行实验脚本时，应优先在 `daran/` 或其子目录下执行，以保证相对路径中的模型文件、字体文件和标定结果可以正确加载。

1.3 硬件连接与安全约束

机械臂通过串口转 CAN 模块与上位机通信，默认波特率为 115200。代码中常见串口名称包括 Windows 下的 `COM4/COM7`，Linux 或 Jetson 下的 `/dev/ttyUSB0`，以及

部分脚本中使用的 `/dev/serial/by-id/...`。由于 `arm_robot` 对象在实例化后会打开串口并控制真实硬件，实验中必须区分仿真脚本和实机脚本。

本仓库形成了明确的“先仿真、后实机”安全模式。以任务 1 和任务 2 为例，脚本中使用 `MOVE_REAL_ROBOT` 或 `--dry-run/--execute` 这样的开关控制是否真实运动；在打开实机开关前，需要先检查关节限位、桌面高度、目标点可达性和仿真动画。后续报告中的仿真验证均遵守这一原则，不直接执行会驱动真实机械臂的代码。

1.4 仓库结构与实验代码组织

实验代码主要位于 `daran/` 目录下。核心机械臂控制代码采用逐层继承结构：底层 `DrEmpower_can.py` 负责串口/CAN 协议，`arm_six_axis.py` 实现六轴运动学，`gripper.py` 添加夹爪控制，`arm_robot.py` 提供面向实验脚本的机械臂控制接口。

与本报告六个实验对应的主要文件如表 2 所示。

表 2: 实验内容与仓库文件对应关系

实验	主要目录或文件	内容
环境配置	<code>environment.yml</code> , <code>CLAUDE.md</code>	环境、依赖、运行约束
直线运动	<code>daran/task1/</code>	DH 建模、IK、直线轨迹仿真
抓取搬运	<code>daran/task2/pick_and_place.py</code>	示教点回放和夹爪控制
相机标定	<code>daran/task3/camera_calibration.py</code>	棋盘格内参标定
颜色视觉抓取	<code>daran/task3/visual_grasp.py</code> , <code>daran/color_pick_place/</code>	颜色检测、坐标转换、抓取规划
YOLO 视觉抓取	<code>daran/color_pick_place/yolo/</code>	数据集、训练、推理与接口复用

1.5 实验结果与小结

通过环境配置实验，完成了 `conda` 环境、核心依赖、机械臂控制接口和实验目录结构的确认。后续实验均建立在同一个 `robot` 环境和同一套机械臂模型参数之上，因此环境一致性是保证实验结果可复现的前提。安全方面，本报告后续只运行不触发真实硬件动作的仿真、读取和编译命令；涉及真实机械臂的脚本仅作为实现逻辑分析材料。

2 实验二：机械臂末端直线运动控制

2.1 实验目的

本实验要求控制机械臂末端沿笛卡尔空间中的一条直线运动。实验重点是建立机械臂 DH 模型，利用正运动学得到末端位姿，通过笛卡尔插补生成直线路径，并对路径上的每个采样点进行逆运动学求解，最终得到连续的关节角轨迹。

2.2 机械臂运动学模型

实验采用标准 DH 参数建立六自由度机械臂模型。模型在 `daran/task1/dh_cartesian_line_motion.ipynb`、`daran/task2/pick_and_place.py` 和 `daran/task3/visual_grasp.py` 中保持一致，均由 `roboticstoolbox` 的 `DHRobot` 和 `RevoluteDH` 构造。

表 3: 机械臂标准 DH 参数表

关节	a_i/m	α_i/rad	d_i/m	θ_i
1	0	$\pi/2$	0	q_1
2	0.15	0	0	q_2
3	0.15	0	0	q_3
4	0	$\pi/2$	-0.05494	$q_4 + \pi/2$
5	0	$-\pi/2$	0.068	q_5
6	0	0	0.033	q_6

关节角限位设置为：

$$q_{\min} = [-160, -40, -160, -160, -180, -180]^\circ, \quad q_{\max} = [160, 180, 160, 160, 180, 180]^\circ.$$

在每次轨迹生成后，程序都会检查所有关节角是否位于该范围内，并检查各 DH 坐标系原点是否高于桌面安全线。

2.3 直线轨迹规划方法

直线运动实验的起点采用安全竖直 park 姿态：

$$q_{\text{park}} = [0, 90, 0, 0, 0, 0]^\circ.$$

在 notebook 中，起点由正运动学得到，终点通过 `path_end_mm` 指定。程序调用 `rtb.tools.trajectory.ctraj` 在起止末端位姿之间生成笛卡尔路径，再对每个路径点使用

ikine_LM 求逆运动学。为了保证关节空间轨迹连续，第 k 个点的逆解会作为第 $k + 1$ 个点的初值。

路径生成可概括为：

$$T_k = \text{ctrj}(T_0, T_1, N)_k, \quad q_k = \text{IK}(T_k, q_{k-1}).$$

其中 T_k 为第 k 个末端位姿， q_k 为对应关节角， N 为轨迹点数。

2.4 程序实现

核心程序位于 `daran/task1/dh_cartesian_line_motion.ipynb`。该 notebook 的主要步骤包括：

1. 使用 DH 参数和关节限位创建 `DFarm_StdDH` 模型。
2. 设置起点 `park` 姿态和终点 `path_end_mm`。
3. 使用 `ctrj` 生成笛卡尔直线轨迹。
4. 对每个轨迹点调用 `ikine_LM` 求解关节角。
5. 计算实际末端位置与目标点的残差。
6. 生成末端轨迹动画和关节角变化图。

notebook 输出显示，轨迹点数为 40，最大 IK 位置残差约为 0.636 mm。该误差相对桌面机械臂的操作尺度较小，说明逆运动学求解能够较好跟踪笛卡尔插补路径。

关键代码如代码 2 所示，主要体现了 DH 模型构建、`ctrj` 笛卡尔插补和逐点逆运动学求解。

Listing 2: 末端直线运动轨迹生成与逆运动学求解

```

1  from roboticstoolbox import DHRobot, RevoluteDH
2  import roboticstoolbox as rtb
3  import numpy as np
4
5  QMIN_DEG = [-160, -40, -160, -160, -180, -180]
6  QMAX_DEG = [160, 180, 160, 160, 180, 180]
7
8  LINK_PARAMS = (
9      dict(alpha=np.pi / 2),
10     dict(a=0.15),

```

```

11     dict(a=0.15),
12     dict(d=-0.05494, alpha=np.pi / 2, offset=np.pi / 2),
13     dict(d=0.068, alpha=-np.pi / 2),
14     dict(d=0.033),
15 )
16
17
18 def make_dfarm() -> DHRobot:
19     links = [
20         RevoluteDH(qlim=np.deg2rad([QMIN_DEG[i], QMAX_DEG[i]]), **kw)
21         for i, kw in enumerate(LINK_PARAMS)
22     ]
23     return DHRobot(links, name="DFarm_StdDH")
24
25
26 dfarm = make_dfarm()
27 q_start_deg = np.array([0, 90, 0, 0, 0, 0], dtype=float)
28 path_end_mm = np.array([100, 200, 200], dtype=float)
29 waypoint_count = 40
30
31 T_start = dfarm.fkine(np.deg2rad(q_start_deg))
32 T_end = T_start.copy()
33 T_end.t = path_end_mm / 1000.0
34 cartesian_path = rtb.tools.trajectory.ctraj(T_start, T_end, waypoint_count
    ↪ )
35
36 joint_path_deg = []
37 q_seed = np.deg2rad(q_start_deg)
38 for idx, T_step in enumerate(cartesian_path):
39     ik = dfarm.ikine_LM(T_step, q0=q_seed, joint_limits=True, ilimit=5000)
40     if not ik.success:
41         raise RuntimeError(f"IK failed at waypoint {idx}: {ik.reason}")
42     q_seed = ik.q
43     joint_path_deg.append(np.rad2deg(ik.q))

```

2.5 仿真与实验结果

图 1 展示了单目标位姿仿真结果，图 2 展示了对应关节角变化曲线。仿真结果表明，机械臂可以从初始安全姿态运动到目标末端位姿，且关节角变化处于限位范围内。

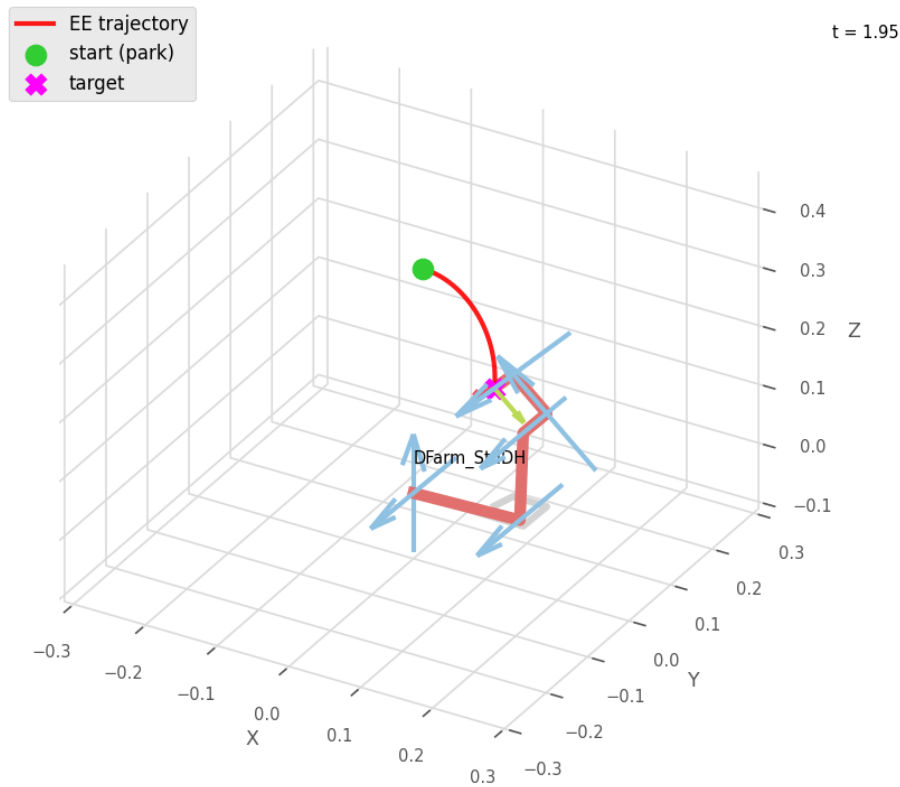


图 1: 机械臂目标位姿仿真结果

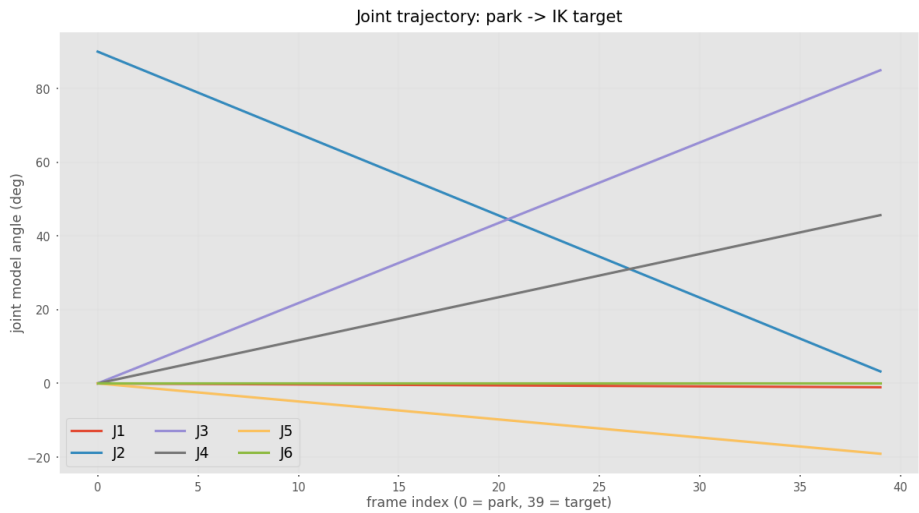


图 2: 目标运动过程中的关节角变化

2.6 结果分析与小结

本实验验证了 DH 建模、笛卡尔插补和数值逆运动学组合使用的可行性。相比直接在关节空间线性插值，笛卡尔空间插补可以更直接约束末端轨迹形状；但其代价是

每个路径点都需要进行逆运动学求解，并且可能受初值、关节限位和奇异位形影响。因此程序中采用前一点解作为下一点初值，并在仿真阶段进行限位和桌面高度检查，为后续抓取搬运提供了安全的运动规划基础。

3 实验三：机械臂抓取与搬运

3.1 实验目的

本实验要求控制机械臂从一点抓取物体，并搬运到另一点放置。与单纯的末端轨迹控制相比，抓取搬运任务需要同时处理关键位姿序列、夹爪开合、桌面安全高度和实机执行保护。

3.2 任务流程设计

程序位于 `daran/task2/pick_and_place.py`，采用示教点回放方式完成 A 点抓取和 B 点放置。示教文件 `ref_pose.json` 中记录了四个关键点：

表 4: 抓取搬运实验示教点

顺序	名称	含义
1	A_safe	A 点上方安全位
2	A_grasp	A 点抓取位
3	B_safe	B 点上方安全位
4	B_place	B 点放置位

完整执行序列为：

$$\text{park} \rightarrow A_{\text{safe}} \rightarrow A_{\text{grasp}} \rightarrow A_{\text{safe}} \rightarrow B_{\text{safe}} \rightarrow B_{\text{place}} \rightarrow B_{\text{safe}} \rightarrow \text{park}.$$

其中到达 A 点抓取位后闭合夹爪，到达 B 点放置位后打开夹爪。

3.3 关键位姿与夹爪控制

脚本定义了固定 `park` 位姿 $[0, 90, 0, 0, 0, 0]^\circ$ 作为任务起止安全姿态。示教点中的关节角全部经过限位检查，A/B 安全点还需满足末端高度不低于 130 mm，safe 点到 grasp/place 点的下降幅度不得超过 120 mm。

夹爪控制未直接使用原始 `grasp()` 接口，而是根据实测关系调用底层 `set_angle_adaptive`：

$$\theta_g = w \cdot \frac{-180}{\pi d},$$

其中 w 为夹爪开口宽度, $d = 10\text{ mm}$ 为夹爪传动直径。实验中张开宽度为 6.0 mm , 夹紧宽度为 1.8 mm , 闭爪力约为 40 N 。

3.4 程序实现

抓取搬运程序沿用了实验二中的 DH 模型和关节限位。主要函数包括：

- `load_waypoints()`: 读取示教点并检查数据长度和关节限位。
- `safety_check_reference_set()`: 计算示教点正运动学位置, 检查桌面高度、安全点高度和下降幅度。
- `build_plan()`: 构造移动和夹爪动作序列。
- `verify_ee_z_floor()`: 检查整条轨迹末端高度下限。
- `MOVE_REAL_ROBOT`: 控制是否真实下发串口指令。

值得注意的是, 脚本头部注释要求先将 `MOVE_REAL_ROBOT` 设为 `False` 生成仿真动画, 确认无误后再由操作者手动改为 `True` 执行实机。这一模式降低了误操作导致碰撞的风险。

抓取搬运的核心动作序列和夹爪宽度到角度的换算如代码 3 所示。代码中将移动动作和夹爪动作放在同一计划列表中, 便于仿真和实机执行共用同一套流程。

Listing 3: 抓取搬运动作序列与夹爪宽度换算

```

1  import numpy as np
2
3  PARK_POSE_DEG = [0, 90, 0, 0, 0, 0]
4  GRIPPER_OPEN_WIDTH_MM = 6.0
5  GRIPPER_CLOSE_WIDTH_MM = 1.8
6  GRIPPER_DEG_PER_MM = -180.0 / (np.pi * 10.0)
7
8
9  def build_plan(waypoints):
10     A_safe = waypoints[0][1]
11     A_grasp = waypoints[1][1]
12     B_safe = waypoints[2][1]
13     B_place = waypoints[3][1]
14
15     return [
16         ("park -> A_safe", "move", PARK_POSE_DEG, A_safe),

```

```

17     ("open gripper", "gripper", GRIPPER_OPEN_WIDTH_MM, None),
18     ("A_safe -> A_grasp", "move", A_safe, A_grasp),
19     ("close gripper", "gripper", GRIPPER_CLOSE_WIDTH_MM, None),
20     ("A_grasp -> A_safe", "move", A_grasp, A_safe),
21     ("A_safe -> B_safe", "move", A_safe, B_safe),
22     ("B_safe -> B_place", "move", B_safe, B_place),
23     ("open gripper", "gripper", GRIPPER_OPEN_WIDTH_MM, None),
24     ("B_place -> B_safe", "move", B_place, B_safe),
25     ("B_safe -> park", "move", B_safe, PARK_POSE_DEG),
26 ]
27
28
29 def gripper_width_to_angle(width_mm: float) -> float:
30     return float(width_mm) * GRIPPER_DEG_PER_MM

```

3.5 实验结果与分析

图 3 为抓取搬运动画中抽取的一帧。实验流程中，机械臂先回到竖直 park 姿态，再移动到 A 点上方；下降到抓取位后闭合夹爪，随后抬升、平移至 B 点上方，再下降放置并打开夹爪。该流程避免了物体在桌面附近长距离水平移动，从而降低夹爪、物体和桌面发生干涉的可能性。

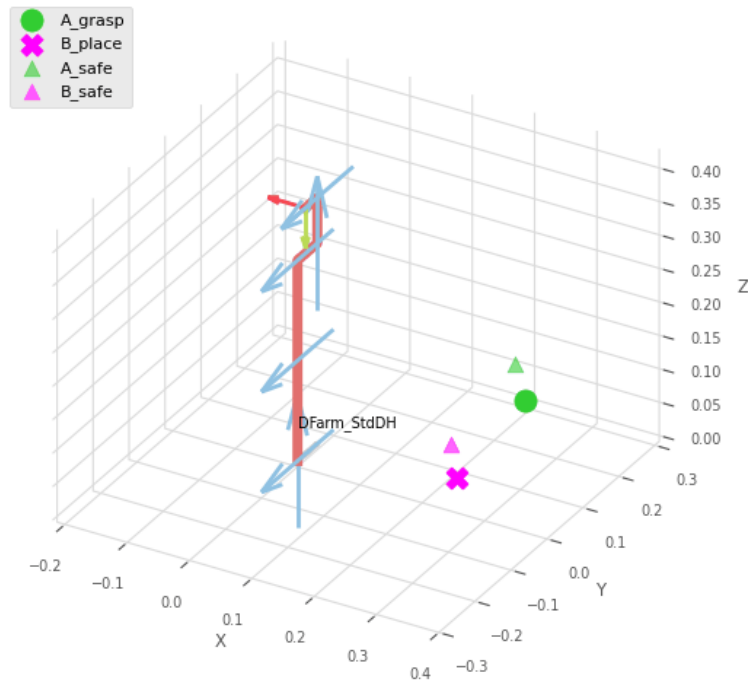


图 3: 抓取搬运动画静态帧

实验中四个示教点由实机示教得到，示教时间为 2026 年 5 月 14 日。A 点和 B 点均设置 safe/grasp 或 safe/place 两级高度，体现出抓取任务中“先到上方、再垂直下降”的基本安全策略。

3.6 小结

本实验将前一实验中的轨迹规划扩展为包含夹爪动作的任务级流程。其关键不是单个点的运动，而是多个安全位姿之间的顺序约束。后续视觉抓取实验本质上也是先通过视觉确定抓取点，再自动生成类似的 safe-grasp-safe 序列，因此本实验为视觉抓取提供了动作模板。

4 实验四：相机标定

4.1 实验目的

本实验目标是获得相机内参、畸变参数以及相机与工作平面之间的位姿关系，为后续从图像像素坐标推算机械臂抓取坐标提供基础。没有可靠的相机标定，视觉识别只能得到像素位置，无法直接用于机械臂运动控制。

4.2 相机成像模型

实验采用针孔相机模型。空间点 $P_C = [X_C, Y_C, Z_C]^T$ 在相机坐标系下投影到像素平面：

$$s \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = K \begin{bmatrix} X_C \\ Y_C \\ Z_C \end{bmatrix}, \quad K = \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix}.$$

其中 K 为相机内参矩阵， f_x, f_y 为像素焦距， c_x, c_y 为主点坐标。实际镜头还存在径向畸变和切向畸变，需要通过标定估计畸变系数。

4.3 内参与畸变标定

内参标定程序为 `daran/task3/camera_calibration.py`。程序支持实时采集和从已有图片标定两种方式。实验使用 7×7 内角点棋盘格，方格边长为 0.008 m，有效标定图像共 30 张。程序流程如下：

1. 使用 `cv2.findChessboardCorners` 检测棋盘格角点。
2. 使用 `cv2.cornerSubPix` 对角点进行亚像素优化。

3. 构造棋盘格世界坐标点。
4. 调用 `cv2.calibrateCamera` 求解内参和畸变。
5. 保存 `camera_calibration.json`、`camera_calibration.npz` 和去畸变示例图。

标定结果如表 5 所示。

表 5: 相机内参标定结果

项目	数值
图像尺寸	640×480
有效图像数	30
RMS 重投影误差	0.2025 px
平均重投影误差	0.2025 px
f_x	381.2781
f_y	508.3521
c_x	319.2541
c_y	239.6599

标定程序中棋盘格三维点生成、角点检测和相机标定调用如代码 4 所示。该部分是后续所有像素坐标转换的基础。

Listing 4: 棋盘格角点检测与相机内参标定核心代码

```

1  import cv2
2  import numpy as np
3
4
5  def make_object_points(cols, rows, square_size):
6      objp = np.zeros((rows * cols, 3), np.float32)
7      objp[:, :2] = np.mgrid[0:cols, 0:rows].T.reshape(-1, 2)
8      return objp * square_size
9
10
11 def find_corners(gray, pattern_size):
12     flags = cv2.CALIB_CB_ADAPTIVE_THRESH + cv2.CALIB_CB_NORMALIZE_IMAGE
13     ok, corners = cv2.findChessboardCorners(gray, pattern_size, flags)
14     if not ok:
15         return False, None
16

```

```
17     criteria = (cv2.TERM_CRITERIA_EPS + cv2.TERM_CRITERIA_MAX_ITER, 30,  
    ↪ 0.001)  
18     corners = cv2.cornerSubPix(gray, corners, (11, 11), (-1, -1), criteria  
    ↪ )  
19     return True, corners  
20  
21  
22 def calibrate_camera(objpoints, imgpoints, image_size):  
23     rms, camera_matrix, dist_coeffs, rvecs, tvecs = cv2.calibrateCamera(  
24         objpoints, imgpoints, image_size, None, None  
25     )  
26     new_camera_matrix, roi = cv2.getOptimalNewCameraMatrix(  
27         camera_matrix, dist_coeffs, image_size, 1, image_size  
28     )  
29     return rms, camera_matrix, dist_coeffs, new_camera_matrix, roi, rvecs,  
    ↪ tvecs
```

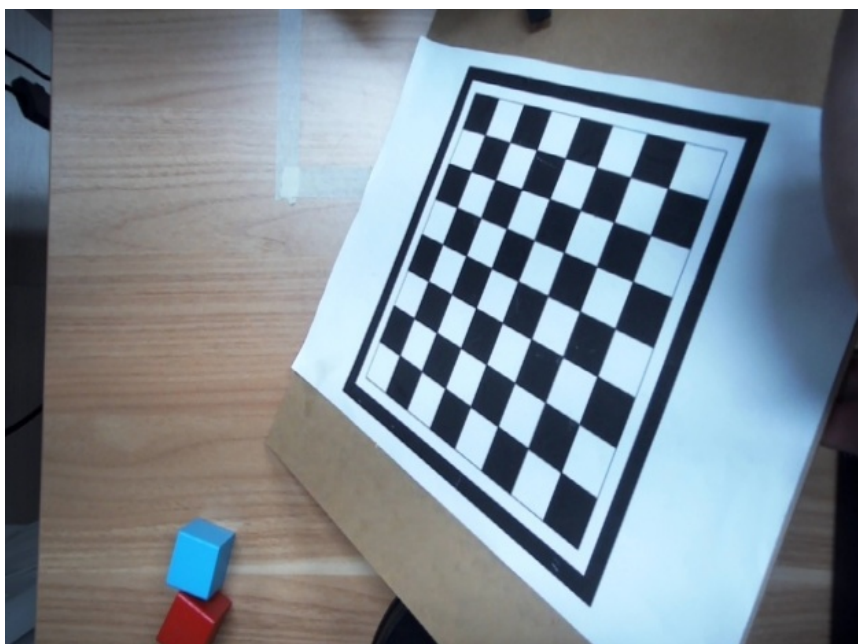


图 4: 相机去畸变示例图

4.4 工作空间坐标转换

外参估计程序为 `daran/task3/pose_projection.py`。在工作平面上放置棋盘格后,程序通过 `cv2.solvePnP` 求解世界坐标系到相机坐标系的变换矩阵,并保存 `camera_extrinsic.json`。实验得到的外参重投影误差约为 0.0215 px。

对于一个像素点 (u, v) ，程序先使用内参和畸变参数通过 `cv2.undistortPoints` 得到相机归一化射线，再将射线变换到世界坐标系，并与工作平面 $z = z_0$ 求交：

$$P_W = O_W + \lambda d_W, \quad \lambda = \frac{z_0 - O_{W,z}}{d_{W,z}}.$$

在视觉抓取实验中，还需要将棋盘格或相机工作平面坐标转换到机械臂基坐标。当前 `workspace_transform.json` 中采用的轴关系为：

$$x_{\text{base}} = -y_{\text{cam}} + 0.300, \quad y_{\text{base}} = x_{\text{cam}} - 0.028, \quad z_{\text{base}} = z_{\text{cam}}.$$



图 5: 棋盘格外参估计与坐标轴投影

4.5 标定结果与分析

从内参标定结果看，平均重投影误差约为 0.20 px，说明棋盘格角点拟合精度较高。外参估计的重投影误差更低，约为 0.0215 px，表明在单幅棋盘格图像上估计得到的位姿与角点观测匹配良好。

但需要注意，图像重投影误差小并不等价于机械臂抓取误差一定小。抓取误差还会受到以下因素影响：

- 相机安装位置和工作平面是否稳定；
- 棋盘格平面与真实物体上表面是否存在高度差；
- 像素中心提取是否受光照、遮挡和颜色反光影响；
- 机械臂自身运动学模型和舵机回差误差。

4.6 小结

本实验完成了从像素坐标到工作平面坐标的关键标定。后续颜色识别和 YOLO 识别都只是获得目标像素中心，真正能让机械臂去抓取目标的是本实验建立的相机内参、外参和工作空间到机械臂基坐标的转换关系。

5 实验五：基于颜色识别的视觉抓取

5.1 实验目的

本实验在相机标定基础上，通过颜色识别获得物体在图像中的位置，再将像素坐标转换为机械臂基坐标，生成抓取轨迹并完成放置任务。相比前面的示教点抓取，本实验的抓取点由视觉实时获得，体现了从固定动作回放 to 视觉引导动作规划的转变。

5.2 颜色分割与目标定位

颜色识别代码主要位于 `daran/color_pick_place/detect.py`。程序定义了红、绿、蓝、黄四类颜色的 HSV 阈值范围，对图像进行颜色分割后，通过轮廓提取获得候选色块。检测流程如下：

1. 读取相机图像并根据标定参数进行去畸变。
2. 将 BGR 图像转换到 HSV 颜色空间。
3. 根据目标颜色阈值生成二值 mask。
4. 使用开运算和闭运算去除噪声、填补孔洞。
5. 通过 `cv2.findContours` 提取轮廓。
6. 根据面积阈值和长宽比过滤非方块目标。
7. 使用 `cv2.minAreaRect` 得到目标中心、边界框和旋转角。

对于蓝色和绿色，程序还叠加了 BGR 通道约束。例如蓝色目标要求蓝色通道显著高于红色和绿色通道，以增强在复杂光照下的鲁棒性。

颜色阈值分割和轮廓提取的核心代码如代码 5 所示。该代码保留了原实验中“HSV 阈值 + 形态学处理 + 最小外接矩形”的基本流程。

Listing 5: 颜色阈值分割与色块轮廓提取

```
1 import cv2
```

```
2 import numpy as np
3
4 COLOR_RANGES = {
5     "red": ([0, 100, 100], [10, 255, 255]), ([160, 100, 100], [179, 255,
6         ↪ 255]]),
7     "green": ([35, 50, 50], [85, 255, 255]),
8     "blue": ([78, 70, 35], [135, 255, 255]),
9     "yellow": ([20, 100, 100], [30, 255, 255]),
10 }
11
12 def build_color_mask(image, color_name):
13     hsv = cv2.cvtColor(image, cv2.COLOR_BGR2HSV)
14     mask = np.zeros_like(hsv[:, :, 0])
15     for lower, upper in COLOR_RANGES[color_name]:
16         mask |= cv2.inRange(hsv, np.array(lower, np.uint8), np.array(upper
17             ↪ , np.uint8))
18
19     kernel = np.ones((5, 5), np.uint8)
20     mask = cv2.morphologyEx(mask, cv2.MORPH_OPEN, kernel)
21     mask = cv2.morphologyEx(mask, cv2.MORPH_CLOSE, kernel, iterations=2)
22     return mask
23
24 def detect_blocks(image, color_name, min_area=500.0):
25     mask = build_color_mask(image, color_name)
26     contours, _ = cv2.findContours(mask, cv2.RETR_EXTERNAL, cv2.
27         ↪ CHAIN_APPROX_SIMPLE)
28     detections = []
29     for contour in contours:
30         area = cv2.contourArea(contour)
31         if area <= min_area:
32             continue
33         rect = cv2.minAreaRect(contour)
34         (cx, cy), (rw, rh), angle = rect
35         if rw <= 0 or rh <= 0:
36             continue
37         detections.append({"type": color_name, "center": (cx, cy), "angle"
38             ↪ : angle})
39     return detections
```

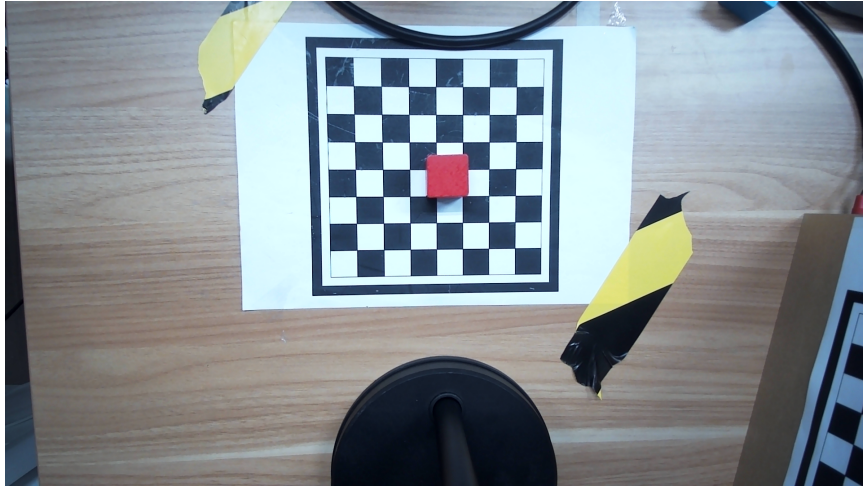


图 6: 颜色识别实验采集图像示例

5.3 像素坐标到机械臂坐标转换

坐标转换由 `daran/color_pick_place/convert.py` 实现。检测结果可以是像素点 (u, v) ，也可以包含角度信息的字典：

$$\{\text{center} : (u, v), \text{angle} : \theta\}.$$

转换步骤如下：

1. 读取 `camera_calibration_result.json` 中的相机内参和畸变参数。
2. 读取 `extrinsics.json` 中相机到机械臂基坐标的齐次变换 $T_{\text{robot}}^{\text{camera}}$ 。
3. 使用 `cv2.undistortPoints` 将像素点转换为相机归一化射线。
4. 将射线变换到机械臂坐标系，并与桌面平面 $z = 80 \text{ mm}$ 求交。
5. 输出机械臂坐标 (x, y, z) 和抓取姿态 $(\text{pitch}, \text{yaw}, \text{roll})$ 。

色块角度用于调整末端 `yaw`。程序采用 `angle_gain=-1` 和方块角度折返函数，将图像中的平面旋转角映射到夹爪绕竖直方向的角度修正。

像素射线与桌面平面求交的实现如代码 6 所示。它先将像素点去畸变为相机归一化射线，再通过相机到机械臂坐标变换求出机器人坐标系下的桌面交点。

Listing 6: 像素点转换到机械臂桌面坐标的核心代码

```

1 import cv2
2 import numpy as np
3

```

```

4
5 def camera_observation_to_robot_xy(pixel, table_z_mm, camera_matrix,
   ↪ dist_coeffs, T_robot_camera):
6     u, v = pixel
7     rotation_robot_camera = T_robot_camera[:3, :3]
8     translation_robot_camera = T_robot_camera[:3, 3]
9
10    pts = np.array([[u, v]], dtype=np.float64)
11    undistorted = cv2.undistortPoints(pts, camera_matrix, dist_coeffs)
12    x_norm, y_norm = undistorted[0, 0]
13
14    ray_camera = np.array([x_norm, y_norm, 1.0], dtype=np.float64)
15    ray_robot = rotation_robot_camera @ ray_camera
16    if abs(ray_robot[2]) < 1e-9:
17        raise RuntimeError("camera ray is parallel to the robot table
   ↪ plane")
18
19    scale = (table_z_mm - translation_robot_camera[2]) / ray_robot[2]
20    point_robot = rotation_robot_camera @ (scale * ray_camera) +
   ↪ translation_robot_camera
21    return float(point_robot[0]), float(point_robot[1])

```

5.4 抓取流程设计

完整抓取流程由 `daran/color_pick_place/main.py` 实现。程序先构造检测器，再进入“识别、选择、规划、仿真、执行”的循环。颜色识别模式下，检测器为 `ColorBlockDetector`；程序会过滤已经在目标区域附近的物块，选择需要搬运的目标，然后构造 `pick` 和 `place` 两个 `RobotPose`。

抓取动作序列为：

`pick_up` → `pick_down` → `pick_up` → `transit` → `place_up` → `place_down` → `place_up`.

其中每个位姿都包含末端位置、姿态和夹爪开合状态。程序调用 `JointActionMotion.solve()` 将位姿动作转换为关节动作，再先进行仿真预览，确认后才执行实机运动。

5.5 实验结果与分析

图 7 展示了 `task3` 中锁定视觉抓取目标后的图像结果。颜色识别方法实现简单、推理速度快，不需要训练数据；但它依赖 HSV 阈值和光照条件，且当背景中存在相近颜

色、阴影或反光时容易产生误检。

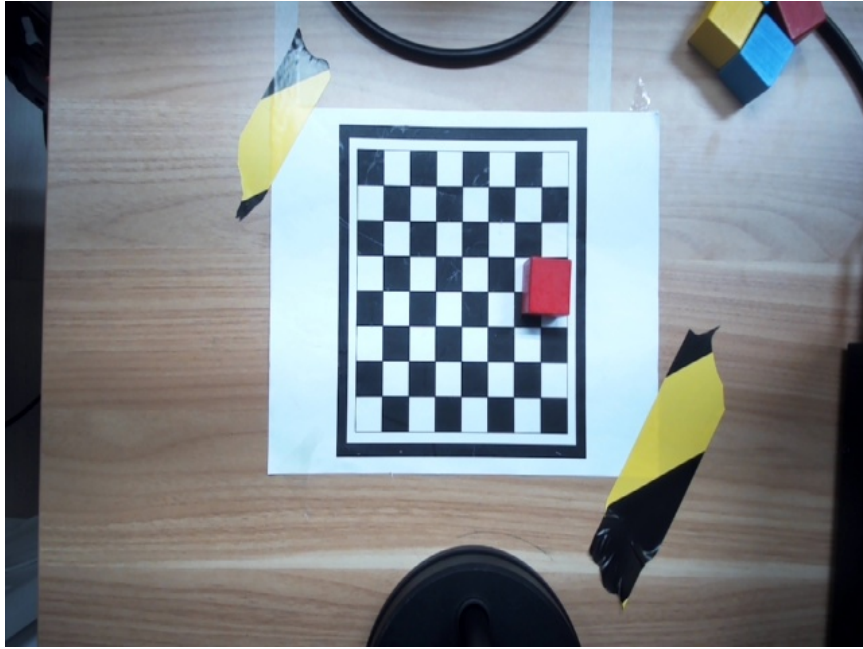


图 7: 颜色视觉抓取目标锁定结果

实验中还通过人工示教建立了像素到机械臂坐标的经验仿射关系, `calib_result.json` 中记录的二维仿射矩阵为:

$$\begin{bmatrix} x_r \\ y_r \end{bmatrix} = \begin{bmatrix} 0.0146 & 0.2952 & 140.9101 \\ 0.2440 & 0.0154 & -224.5123 \end{bmatrix} \begin{bmatrix} u \\ v \\ 1 \end{bmatrix},$$

其 RMS 误差约为 14.10 mm。该结果说明, 经验标定能快速建立视觉到抓取坐标的映射, 但精度会受示教点数量、示教误差和工作平面线性假设影响。

5.6 小结

基于颜色识别的视觉抓取完成了从图像目标到机械臂动作的闭环链路。该方法适合颜色明显、光照稳定、背景简单的桌面场景; 但当目标类别增多或颜色受环境影响明显时, 单纯 HSV 阈值需要频繁调参。因此实验六进一步引入 YOLO, 将颜色阈值识别替换为训练得到的目标检测模型。

6 实验六：基于 YOLO 的视觉抓取

6.1 实验目的

本实验在颜色视觉抓取的基础上，将目标识别部分替换为 YOLO 检测算法。实验目标不是重新编写整套抓取系统，而是保持后端坐标转换、姿态规划、IK 求解和夹爪控制不变，仅将前端目标检测器从 HSV 阈值分割替换为学习型目标检测模型。

6.2 数据集与类别设计

YOLO 数据位于 `daran/color_pick_place/yolo/`。数据集类别包含 red、green、blue 三类色块。根据 `bootstrap_summary.json`，数据集中共有 213 张图像、1041 个标注框，各类别数量如表 6 所示。

表 6: YOLO 数据集统计

类别	标注框数量	占比
red	391	37.6%
green	320	30.7%
blue	330	31.7%
合计	1041	100%

`data.yaml` 指定训练集和验证集目录，并定义类别名称。需要注意的是，该文件中的数据集路径为原训练机器上的绝对路径，因此在不同机器上复现实验时需要将 `path` 改为当前仓库下的实际路径。

6.3 YOLO 模型训练与推理

训练脚本为 `daran/color_pick_place/yolo/train.py`。程序从 `yolo11n.pt` 初始化模型，训练 120 个 epoch，输入尺寸为 960，batch size 为 16。由于类别本身就是颜色，训练脚本中特意设置 `hsv_h=0.0`，避免色相增强把红色样本扰动成橙色或其它颜色，从而破坏类别标签。几何增强则保持较强，包括 180° 旋转、水平/垂直翻转、平移、缩放和 mosaic。

训练参数代码如代码 7 所示，其中关闭色相增强是本实验针对“颜色即类别”这一任务特点做出的关键设置。

Listing 7: YOLO11n 色块检测模型训练参数

```
1 from ultralytics import YOLO
```

```
2
3
4 model = YOLO("yolo11n.pt")
5 model.train(
6     data="data.yaml",
7     epochs=120,
8     imgsz=960,
9     batch=16,
10    device=0,
11    seed=0,
12    patience=40,
13    hsv_h=0.0,          # color is the class; do not shift hue
14    hsv_s=0.5,
15    hsv_v=0.4,
16    degrees=180.0,
17    fliplr=0.5,
18    flipud=0.5,
19    translate=0.1,
20    scale=0.5,
21    mosaic=1.0,
22    project="runs",
23    name="blocks",
24    exist_ok=True,
25 )
```

最终训练结果位于 runs/blocks/，权重文件包括 best.pt 和 last.pt。训练曲线和混淆矩阵如图 8 和图 9 所示。最后一个 epoch 的验证指标约为 precision=0.9917、recall=0.9922、mAP50=0.9915、mAP50-95=0.8577。

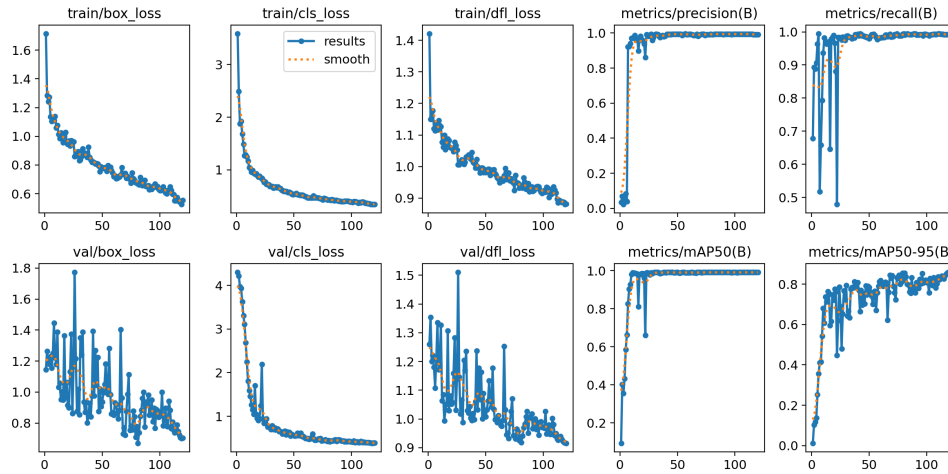


图 8: YOLO 训练指标曲线

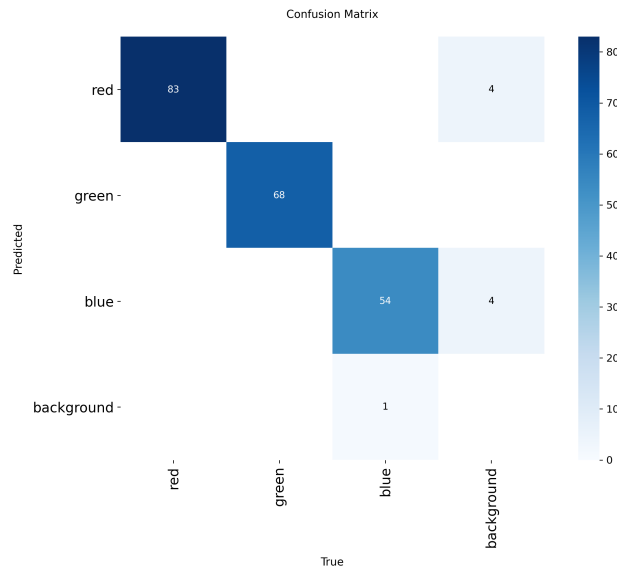


图 9: YOLO 验证集混淆矩阵

6.4 与颜色识别方法的接口复用

YOLO 检测器实现于 `daran/color_pick_place/yolo/yolo_detector.py`。该模块被设计为与 `ColorBlockDetector` 兼容，二者都提供：

Listing 8: 检测器接口约定

```
1 recognize() -> [{"type": str, "center": (u, v), "angle": float}, ...]
2 release()
```

因此 main.py 中只需通过参数选择 `--detector yolo` 或 `--detector color`，后续 `camera_observation_to_robot_pose()`、目标筛选、抓取动作构造和运动执行均可复用。

YOLO 检测器的一个关键设计是：YOLO 负责给出目标类别和边界框，随后仍在检测框 ROI 内调用原颜色 mask 和 `cv2.minAreaRect` 细化中心点和角度。这样做保留了旧系统对 `center` 和 `angle` 的定义，避免后端抓取姿态映射重新标定。

YOLO 检测器的核心推理接口如代码 9 所示。它输出的字典结构与颜色检测器保持一致，因此后端抓取规划不需要区分检测来源。

Listing 9: YOLO 检测器输出与颜色检测器兼容的数据结构

```

1  from ultralytics import YOLO
2  import cv2
3
4
5  class YoloBlockDetector:
6      def __init__(self, weights, conf=0.25, iou=0.5, device="cpu"):
7          self.model = YOLO(weights)
8          self.names = self.model.names
9          self.conf = conf
10         self.iou = iou
11         self.device = device
12
13     def detect_blocks(self, image):
14         result = self.model.predict(
15             image,
16             conf=self.conf,
17             iou=self.iou,
18             device=self.device,
19             verbose=False,
20         )[0]
21
22         detected = []
23         for box in result.boxes:
24             x1, y1, x2, y2 = [int(round(v)) for v in box.xyxy[0].tolist()]
25             color = str(self.names[int(box.cls[0])])
26             conf = float(box.conf[0])
27             cx = (x1 + x2) // 2
28             cy = (y1 + y2) // 2
29             detected.append(

```

```
30         {"type": color, "center": (cx, cy), "angle": 0.0, "conf":
    ↪ conf}
31     )
32     return detected
```

6.5 实验结果与对比分析

图 10 展示了 YOLO 检测和原有颜色逻辑对齐检查的结果。相比颜色阈值方法，YOLO 能利用目标形状、上下文和训练样本分布，在光照变化、局部阴影和颜色接近背景时更稳定；但 YOLO 也引入了训练数据、标注质量、模型权重和推理设备依赖。

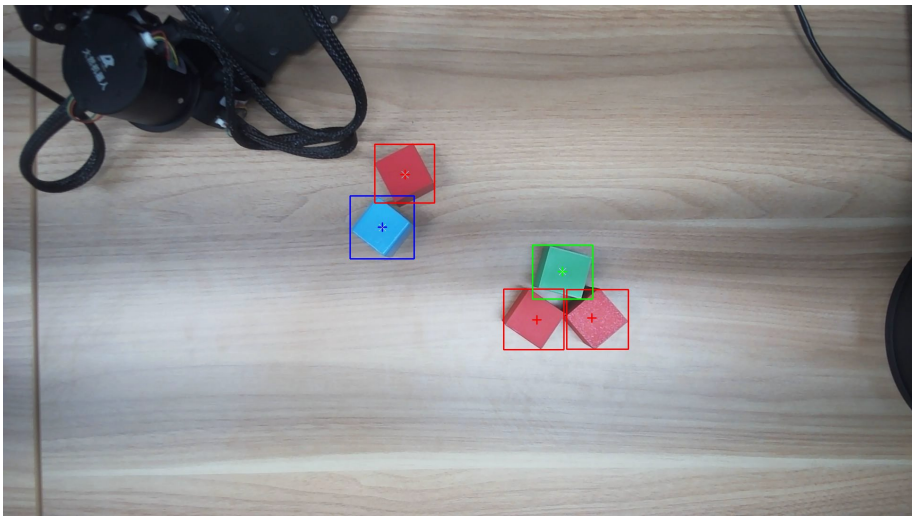


图 10: YOLO 检测结果与颜色逻辑对齐检查

表 7: 颜色识别与 YOLO 识别对比

项目	颜色阈值识别	YOLO 识别
前期成本	无需训练，调阈值即可	需要采集、标注和训练数据
推理速度	很快，CPU 即可	取决于模型和设备
光照适应性	较弱，需调 HSV 阈值	较强，依赖训练集覆盖
可扩展性	类别增多后阈值维护困难	可通过新增类别和再训练扩展
与抓取接口	直接输出中心和角度	输出框后再细化中心和角度

6.6 小结

本实验完成了视觉识别模块的算法升级。通过保持检测器输出接口一致，YOLO 能够无缝接入原有抓取系统，使实验五中的坐标转换、轨迹规划和夹爪控制继续复用。

该设计体现了良好的模块边界：感知前端可以替换，运动控制后端保持稳定。

7 总结与展望

7.1 实验总结

本学期六组实验从基础环境配置开始，逐步完成机械臂运动控制、抓取搬运、相机标定和视觉引导抓取。整体技术路线可以概括为：

环境配置 → 运动学建模 → 轨迹规划
→ 抓取动作 → 相机标定 → 视觉识别 → 视觉抓取.

实验一完成了 Python、Robotics Toolbox、OpenCV、串口通信等基础环境配置；实验二建立 DH 模型并实现末端直线运动；实验三在示教点基础上完成抓取搬运；实验四通过棋盘格标定获得相机内参、畸变参数和工作平面坐标关系；实验五使用颜色分割完成目标定位和视觉抓取；实验六使用 YOLO 替代颜色阈值识别，提高了视觉前端的鲁棒性和可扩展性。

7.2 误差来源分析

综合各实验，系统误差主要来自以下几个方面：

- **运动学模型误差：** DH 参数与真实机械臂结构之间存在制造和装配误差，舵机零位也可能与模型零位不完全一致。
- **执行机构误差：** 一体化关节存在回差、摩擦和负载扰动，夹爪闭合宽度也会受物体材质影响。
- **相机标定误差：** 棋盘格角点检测、相机固定稳定性、工作平面高度假设都会影响像素到世界坐标的转换。
- **视觉检测误差：** 颜色阈值受光照影响明显，YOLO 检测则依赖训练数据覆盖范围和标注质量。
- **任务规划误差：** 抓取高度、下降距离、目标角度估计和放置点高度若设置不合理，会导致抓取不稳或放置偏差。

7.3 改进方向

后续可以从以下方向继续完善系统：

1. 增加手眼标定流程，减少相机坐标到机械臂基坐标之间的人工标定误差。
2. 在抓取前加入二次视觉确认，检测物体是否在夹爪正下方。
3. 引入深度相机或多视角测量，避免单平面高度假设带来的误差。
4. 扩充 YOLO 数据集，加入不同光照、遮挡、背景和物体姿态。
5. 将轨迹规划从关键点插值升级为带速度、加速度约束的更平滑规划。
6. 在真实执行中记录关节反馈和末端误差，形成闭环校正机制。

总体来看，本实验已经形成了一个可运行、可扩展的桌面机械臂视觉抓取系统。它的核心价值不只是完成单个抓取动作，而是建立了从感知、坐标转换、运动规划到执行控制的完整链路，为后续更复杂的自主操作任务打下基础。

参考文献

- [1] Peter Corke. *Robotics Toolbox for Python Documentation*, 2026.
- [2] OpenCV. *OpenCV Documentation*, 2026.
- [3] Ultralytics. *Ultralytics YOLO Documentation*, 2026.